DocuMind RAG

Complete Project Working Details and Interview Guide

Repository: https://github.com/alirazach476/rag-document-qa-agent

Live demo: https://documindrag.streamlit.app/

1. Project Overview

DocuMind RAG is a Retrieval-Augmented Generation (RAG) system that lets users upload a PDF, build a FAISS vector index, and ask questions grounded in that document with source citations. It includes a Streamlit web dashboard and a CLI. Answers are constrained to retrieved context so the model says it does not know when the document does not support an answer.

Goal: demonstrate end-to-end RAG engineering - ingestion, chunking, embeddings, vector search, hybrid retrieval, grounded generation, and dual free deployment (local GPU/Ollama and Streamlit Cloud with Groq).

2. What Problem It Solves

- Users need accurate Q&A over private PDFs without hallucinated answers.
- Classic chatbots invent facts; RAG retrieves real passages first, then generates.
- Supports free local offline use and free cloud demo without OpenAI credits.

3. Tech Stack

- Python 3.10+ - application runtime
- LangChain - RAG orchestration (prompts, documents, runnables)
- FAISS - local persistent vector store
- Sentence-Transformers / HuggingFace - local GPU/CPU embeddings
- FastEmbed - lightweight CPU embeddings for cloud
- Ollama - free local LLM chat
- Groq (OpenAI-compatible API) - free cloud LLM chat
- OpenAI - optional paid provider
- BM25 + custom keyword search - hybrid retrieval with dense vectors
- PyMuPDF - fast PDF text extraction
- Streamlit - DocuMind web dashboard
- pytest / ruff / GitHub Actions - tests, lint, CI

4. End-to-End Working Pipeline

1) Load: PDF pages (or txt/md) become LangChain Documents with source/page metadata.

2) Chunk: RecursiveCharacterTextSplitter (default chunk_size=1200, overlap=100).

3) Embed: Batched embeddings (local MiniLM on CUDA, FastEmbed, Ollama, or OpenAI).

4) Store: FAISS index under vectorstore/<backend>/ or a session temp folder.

5) Retrieve (hybrid): keyword matches first, then BM25, then vector similarity; merge and take top-k.

6) Generate: LLM answers ONLY from context, cites [Source N], else says it does not know.

7) Present: Streamlit chat with expandable sources, or CLI with chunk previews.

Pipeline: docs -> chunk -> embed -> FAISS -> hybrid retrieve -> LLM -> answer + sources

5. Local Free Mode vs Cloud Free Mode

LOCAL (PC): LLM_PROVIDER=ollama (e.g. llama3.2). Embeddings = HuggingFace all-MiniLM-L6-v2 on NVIDIA CUDA when available. Fully free offline after models are pulled.

CLOUD (Streamlit): LLM_PROVIDER=groq with free GROQ_API_KEY. Embeddings = FastEmbed BAAI/bge-small-en-v1.5 on CPU. No OpenAI credits required. Live: https://documindrag.streamlit.app/

Why two modes: Streamlit Cloud cannot run Ollama or your local GPU. Groq + FastEmbed fit free cloud hosting. Separate FAISS folders per embedding backend avoid dimension mismatches.

6. Streamlit Dashboard Behavior

- One PDF per browser session (new upload replaces the previous file).
- PDF bytes kept in session_state; indexing uses a temp workspace.
- Refresh or Remove clears PDF, index, and chat (privacy-friendly for demos).
- Flow: Upload -> Index this PDF -> Ask questions with citations.
- UI theme: DocuMind branded sidebar + hero, status line, chat.

7. Key Engineering Decisions (Say These in Interviews)

- Hybrid retrieval: vectors alone miss exact dictionary words (e.g. Prudent); keyword-first + BM25 fixes lexical misses.
- Grounded prompting: low temperature (0.1); refuse answers outside context.
- Per-backend vector stores: local vs fastembed vs openai indexes stay separate.
- Batched embedding + progress UI: better UX for large PDFs.
- Provider abstraction: ollama | groq | openai with shared RAG core.
- Session isolation on Streamlit: no shared permanent user uploads on cloud.
- Fast PDF parse with PyMuPDF instead of slow pypdf-only path.

8. Project Structure

- app.py - Streamlit DocuMind dashboard
- rag_agent.py / rag_qa/cli.py - CLI entry
- rag_qa/config.py - settings from .env / Streamlit secrets
- rag_qa/providers.py - embeddings + chat models
- rag_qa/ingest.py - document loading
- rag_qa/store.py - chunking + FAISS build/load
- rag_qa/retrieve.py - hybrid keyword + BM25 + vector
- rag_qa/chain.py - ask() + grounded system prompt
- rag_qa/ui_theme.py - dashboard CSS/theme
- tests/ - unit tests for config, ingest, retrieve
- Dockerfile / requirements.txt - cloud and container deploy

9. How to Run (Quick)

Local: install deps, set LLM_PROVIDER=ollama, pull ollama model, streamlit run app.py

Cloud: Streamlit Community Cloud + secrets LLM_PROVIDER=groq, GROQ_API_KEY, EMBEDDING_BACKEND=fastembed

CLI: python rag_agent.py (type questions; rebuild to re-index docs/)

10. Interview Q&A (Memorize These)

Q: What is RAG and how does your project implement it?

A: RAG retrieves relevant document chunks first, then generates an answer using only that context. My project chunks PDFs, embeds them into FAISS, retrieves with hybrid search (keyword + BM25 + vectors), and prompts the LLM to answer with citations or admit it does not know.

Q: Why hybrid retrieval instead of only vector search?

A: Dense embeddings are semantic but can miss exact rare words or dictionary-style entries. I prioritize exact keyword hits, then BM25 lexical ranking, then FAISS similarity, and merge/dedupe results. That fixed cases like 'what is Prudent' where vectors alone returned unrelated chunks.

Q: How do local and cloud free modes differ?

A: Local uses Ollama for chat and HuggingFace MiniLM embeddings on CUDA. Cloud uses Groq free API for chat and FastEmbed on CPU because Streamlit Cloud has no Ollama or GPU. Config auto-selects embedding backend from LLM_PROVIDER.

Q: How do you prevent hallucinations?

A: Strict system prompt: answer only from provided context; cite sources; otherwise say you do not know. Low temperature. Retrieval quality matters as much as the prompt - hybrid search improves context relevance.

Q: Explain your Streamlit one-PDF session design.

A: Uploaded PDF bytes stay in session_state. Indexing writes a temporary docs and vectorstore folder. Refresh or Remove deletes that workspace so each browser session is isolated - good for demos and privacy on a shared cloud app.

Q: Why separate FAISS indexes per embedding backend?

A: Different embedding models produce different vector dimensions and spaces. Loading an OpenAI index with FastEmbed vectors would break similarity search. I store under vectorstore/<backend>/.

Q: How is PDF ingestion implemented?

A: Prefer PyMuPDF for fast per-page text extraction with page metadata; fall back to LangChain PyPDFLoader if needed. Empty image-only pages are skipped. Scanned PDFs without OCR still will not yield text - an honest limitation.

Q: Why not deploy on Vercel?

A: Streamlit needs a long-running Python server and WebSockets. Vercel is for serverless/static frontends. Ollama and GPU also cannot run there. I deployed to Streamlit Community Cloud instead.

Q: What is the difference between ask() and create_rag_chain()?

A: Production Q&A uses ask(), which runs hybrid retrieve_documents then the LLM. create_rag_chain() builds a simpler LCEL chain with FAISS-only retriever - useful for demos, but hybrid ask() is the path used by the UI and CLI.

Q: What would you improve next?

A: Add a cross-encoder reranker, OCR for scanned PDFs, token-aware/semantic chunking, evaluation (faithfulness/citation accuracy), and unify create_rag_chain with hybrid retrieval. For scale, replace full-corpus BM25 scan with a proper inverted index.

11. Limitations (Honest - Interviewers Like This)

- UI is one PDF per session; CLI can index multiple files under docs/.

- Hybrid keyword/BM25 scans the in-memory docstore - fine for one PDF, not huge corpora.
- No OCR - scanned image PDFs extract little text.
- No reranker / multi-query / agent tools - single-shot retrieve-then-generate.
- FAISS load uses allow_dangerous_deserialization for local pickle metadata - only load trusted indexes.
- Cloud quality depends on Groq free tier and a small FastEmbed model.

12. 60-Second Elevator Pitch

I built DocuMind, a RAG document Q&A agent with LangChain and FAISS. Users upload a PDF, I chunk and embed it, retrieve with hybrid keyword plus BM25 plus vector search, then generate grounded answers with citations. It runs free locally with Ollama and GPU embeddings, and free on Streamlit Cloud with Groq and FastEmbed. The live demo is documindrag.streamlit.app.

Prepared for interviews - DocuMind RAG / rag-document-qa-agent